



2^e année cycle ingénieur - Big Data et Machine Learning

RAPPORT DE TRAVAUX PRATIQUES 2

API et Web Services

Création et gestion d'API REST avancées avec Spring Boot

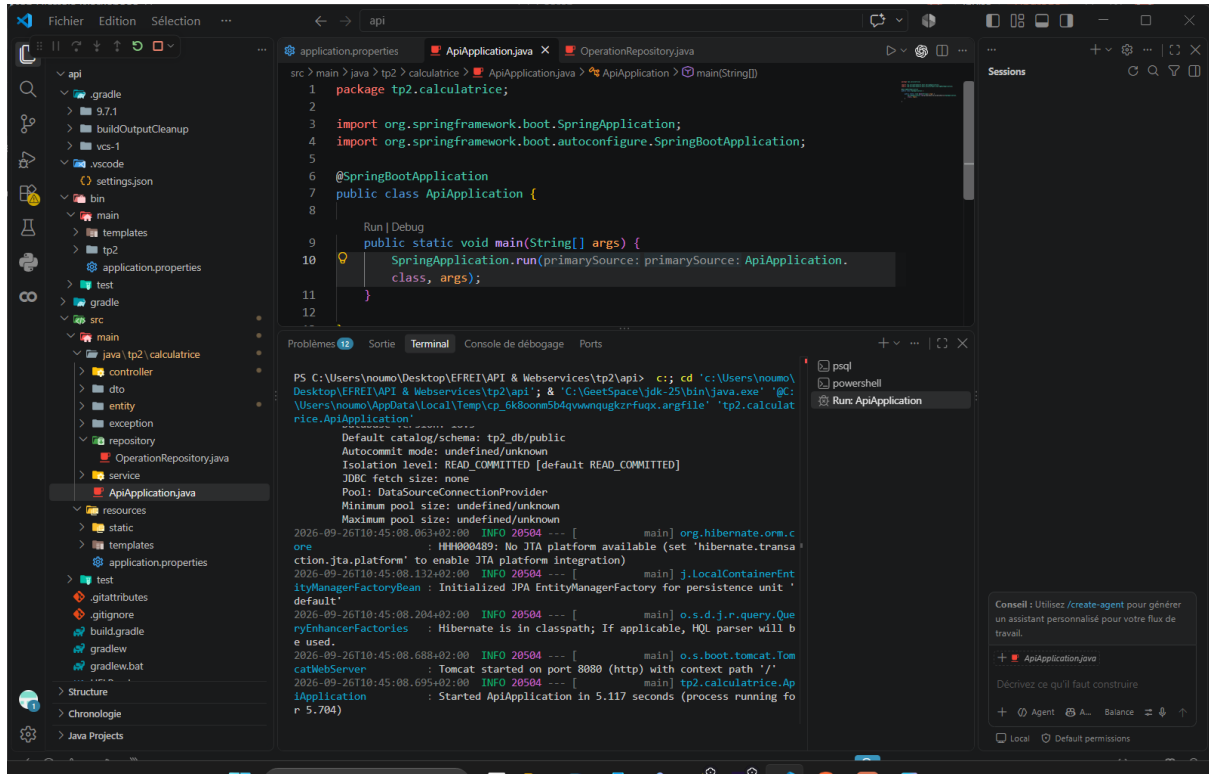
Auteur : Laye Fodé KEITA

Numéro étudiant : 20261720

Année universitaire 2026-2027

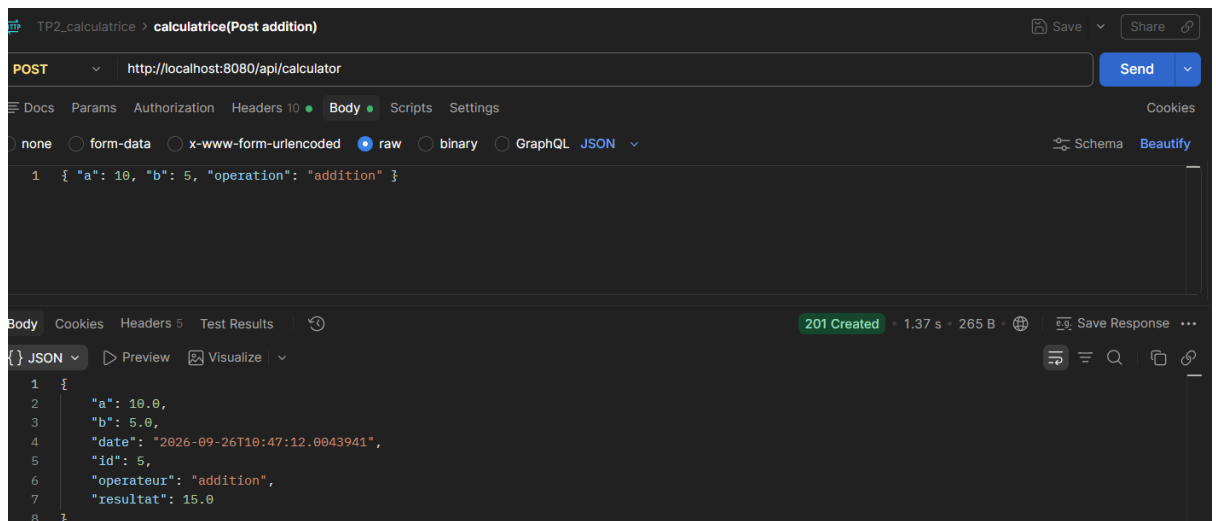
Exercice 1 : API de calculatrice avancée

Configuration du projet Spring Boot. Le projet a été généré avec Spring Initializr (Gradle, Spring Boot 4 et Java 25), avec les dépendances Spring Web, Spring Data JPA, Validation et le pilote PostgreSQL.

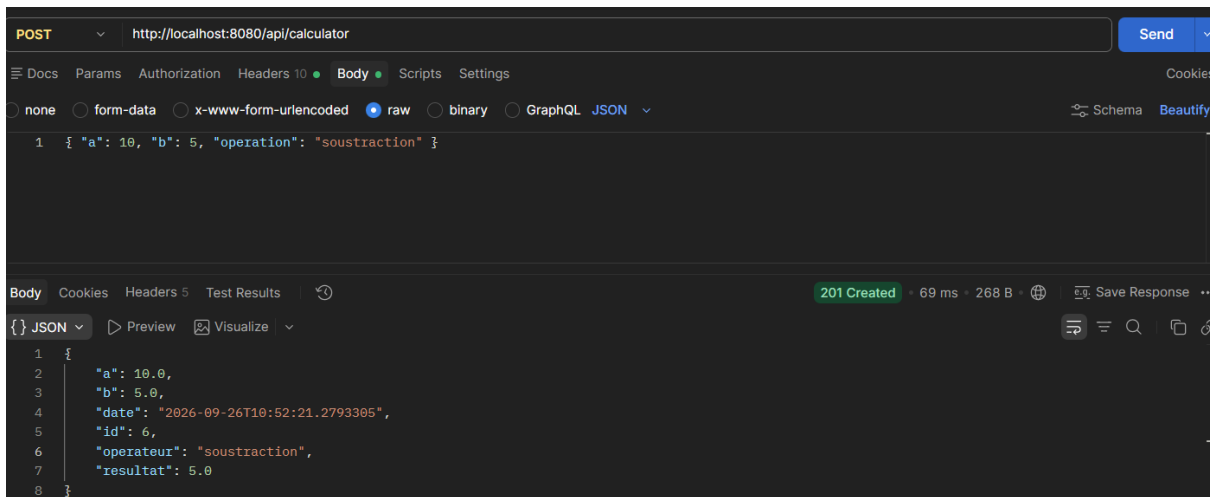


1) Tests dans Postman :

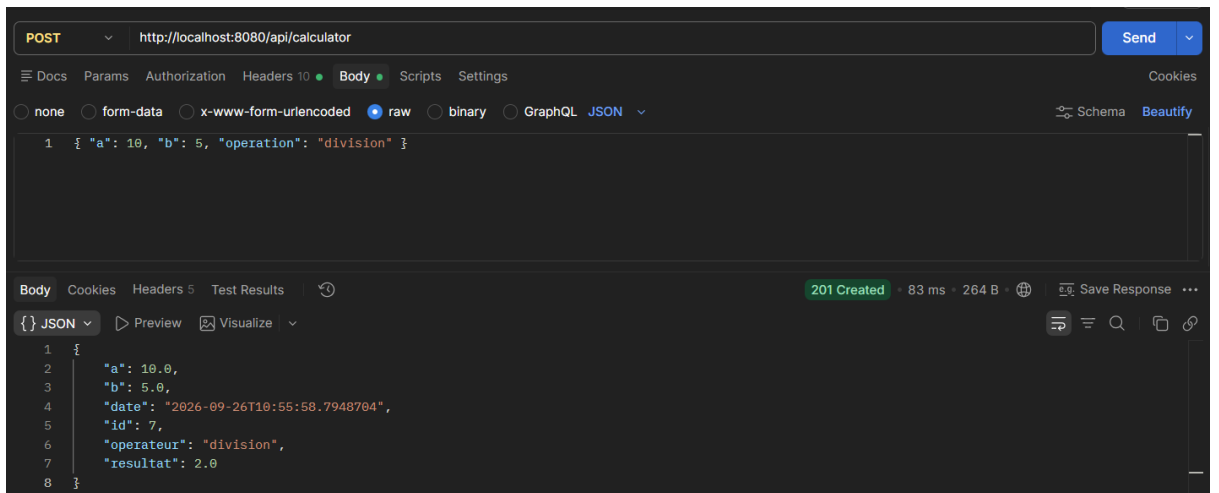
POST : addition de a = 10 et b = 5



POST : soustraction de a = 10 et b = 5



POST : division de $a = 10$ et $b = 5$

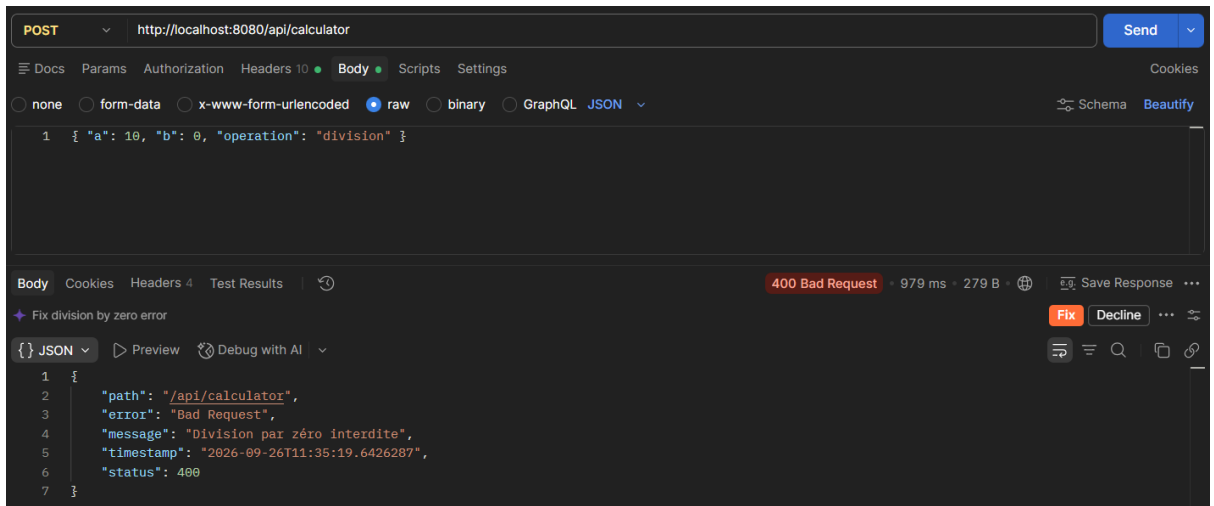


Validation et gestion des erreurs :

Les erreurs sont gérées de façon centralisée dans une classe

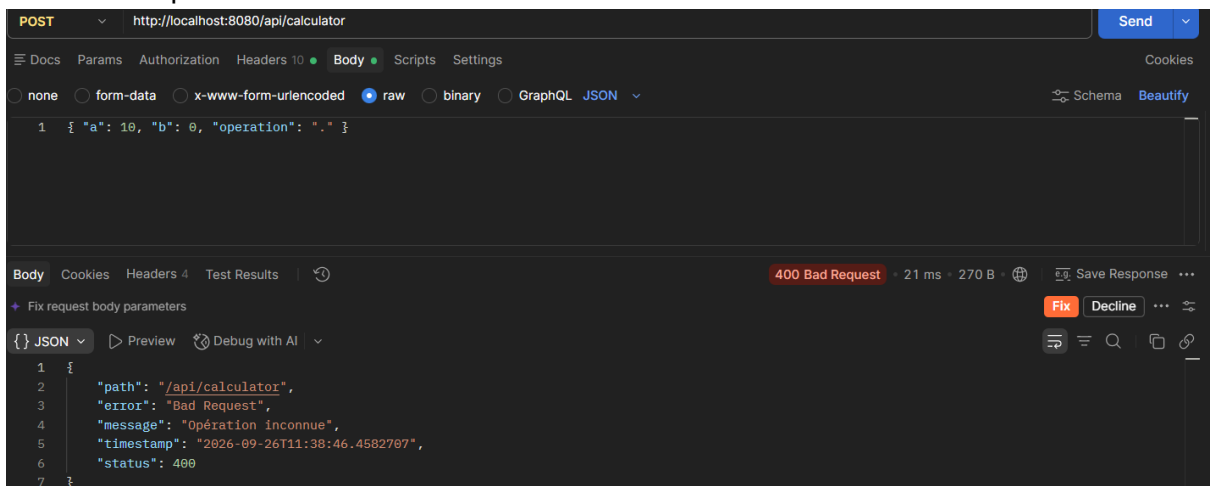
`GlobalExceptionHandler` annotée avec `@RestControllerAdvice`. Ainsi, toutes les erreurs de l'API renvoient un JSON avec le même format (timestamp, status, error, message, path).

Cas de la division par zéro :



L'API renvoie une erreur 400 Bad Request avec un message clair, au lieu de planter avec une exception non gérée.

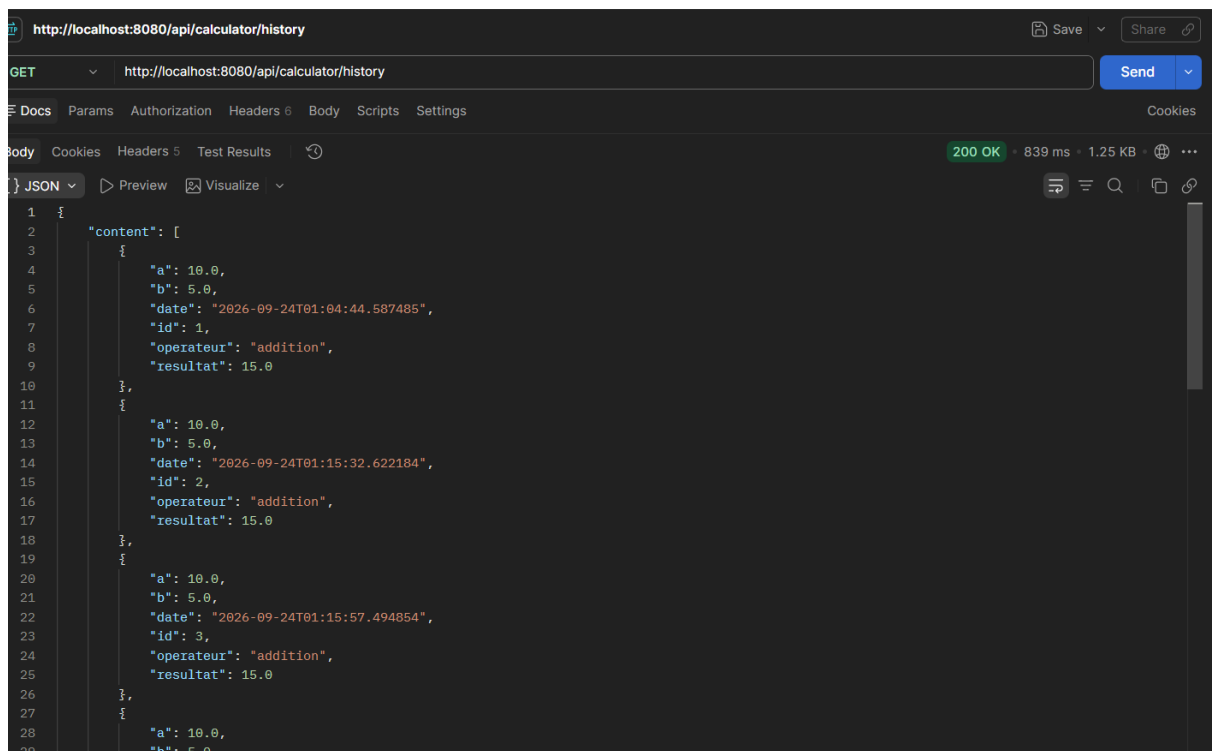
Cas d'une opération inconnue :



CRUD complet :

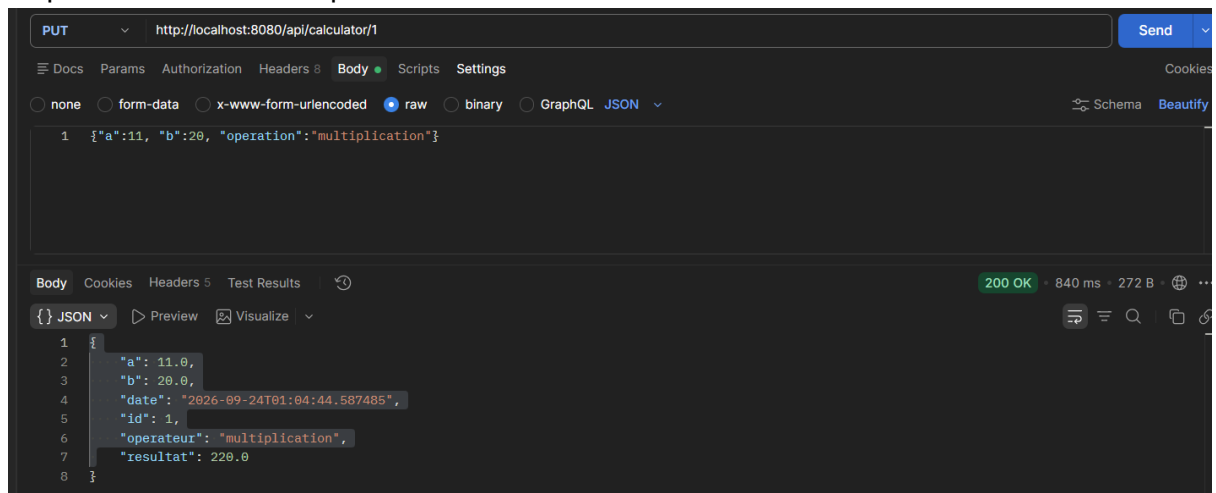
Lister l'historique des opérations avec GET.

<http://localhost:8080/api/calculator/history>



```
1 {
2   "content": [
3     {
4       "a": 10.0,
5       "b": 5.0,
6       "date": "2026-09-24T01:04:44.587485",
7       "id": 1,
8       "opérateur": "addition",
9       "resultat": 15.0
10    },
11    {
12      "a": 10.0,
13      "b": 5.0,
14      "date": "2026-09-24T01:15:32.622184",
15      "id": 2,
16      "opérateur": "addition",
17      "resultat": 15.0
18    },
19    {
20      "a": 10.0,
21      "b": 5.0,
22      "date": "2026-09-24T01:15:57.494854",
23      "id": 3,
24      "opérateur": "addition",
25      "resultat": 15.0
26    }
27  ]
28 }
```

Modifier une opération avec PUT :
<http://localhost:8080/api/calculator/1>

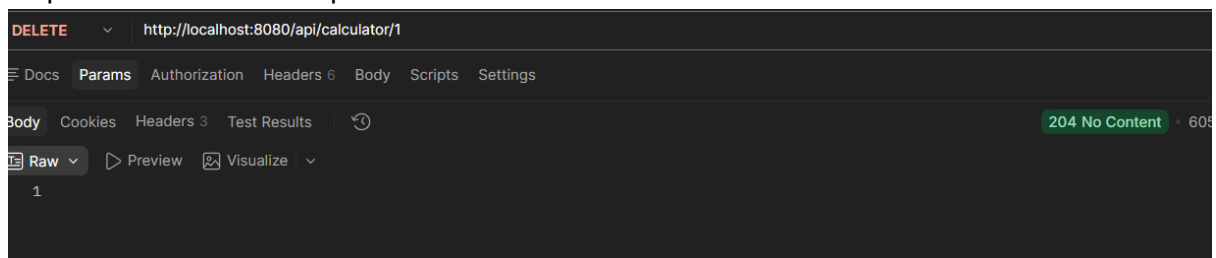


```
1 {"a":11, "b":20, "operation":"multiplication"}
```

```
1 {
2   "a": 11.0,
3   "b": 20.0,
4   "date": "2026-09-24T01:04:44.587485",
5   "id": 1,
6   "opérateur": "multiplication",
7   "resultat": 220.0
8 }
```

Le résultat est recalculé automatiquement avec les nouvelles valeurs ($11 \times 20 = 220$).

Supprimer une opération avec DELETE :
<http://localhost:8080/api/calculator/1>

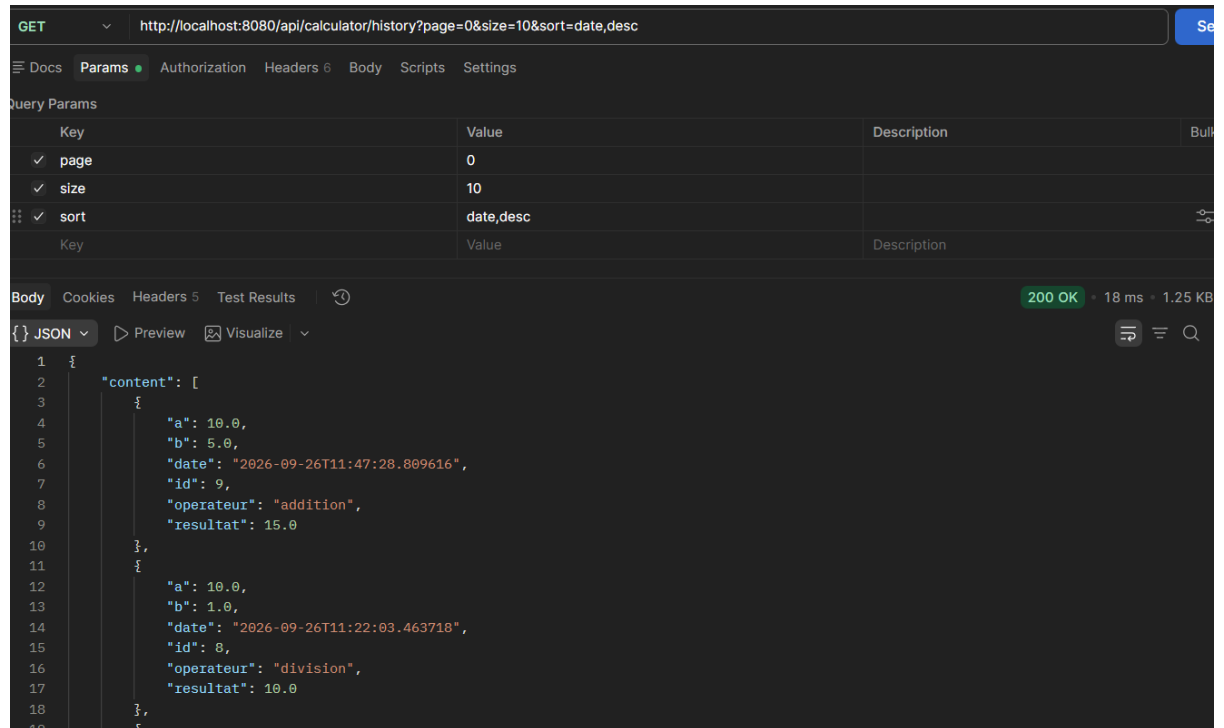


```
1
```

L'opération est supprimée et l'API renvoie le code 204 No Content, sans corps de réponse.

Pagination de l'historique :

<http://localhost:8080/api/calculator/history?page=0&size=10&sort=date,desc>



The screenshot shows a web browser with the address bar displaying the URL: `http://localhost:8080/api/calculator/history?page=0&size=10&sort=date,desc`. The browser's developer tools are open, showing the 'Params' tab with the following query parameters:

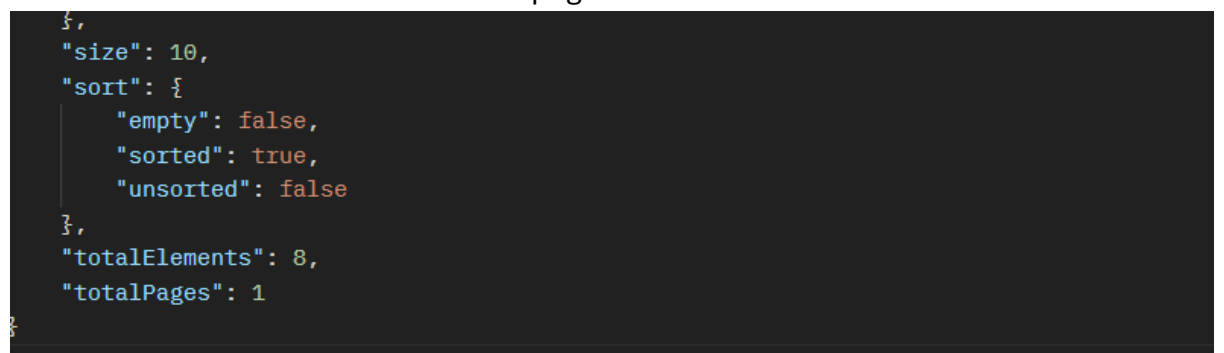
Key	Value	Description
page	0	
size	10	
sort	date,desc	

The 'Body' tab shows the response status as '200 OK' with a response time of '18 ms' and a size of '1.25 KB'. The response body is a JSON object:

```
{
  "content": [
    {
      "a": 10.0,
      "b": 5.0,
      "date": "2026-09-26T11:47:28.889616",
      "id": 9,
      "opérateur": "addition",
      "résultat": 15.0
    },
    {
      "a": 10.0,
      "b": 1.0,
      "date": "2026-09-26T11:22:03.463718",
      "id": 8,
      "opérateur": "division",
      "résultat": 10.0
    }
  ]
}
```

Spring Data construit automatiquement l'objet Pageable à partir des paramètres page, size et sort de l'URL. Cela évite de renvoyer tout l'historique d'un coup quand il devient très grand.

Informations sur les métadonnées de pagination :



The screenshot shows a JSON object representing pagination metadata:

```
{
  "size": 10,
  "sort": {
    "empty": false,
    "sorted": true,
    "unsorted": false
  },
  "totalElements": 8,
  "totalPages": 1
}
```

En plus des opérations, la réponse contient le nombre total d'éléments (totalElements) et le nombre de pages (totalPages).

Structure de la base de données tp2_db sous PostgreSQL :

```

postgres=# \c tp2_db
Vous êtes maintenant connecté à la base de données « tp2_db » en tant qu'utilisateur « postgres ».
tp2_db=# \dt

      Liste des tables
  Schéma |      Nom      | Type | Propriétaire
-----+-----+-----+-----
 public | currency_transaction | table | postgres
 public | exchange_rate      | table | postgres
 public | operation           | table | postgres
(3 lignes)

tp2_db=# desc operation
tp2_db=# \doperation

      Liste des opérateurs
  Schéma | Nom | Type de l'arg. gauche | Type de l'arg. droit | Type du résultat | Description
-----+-----+-----+-----+-----+-----
(0 ligne)

tp2_db=#

```

Les tables sont créées automatiquement par Hibernate à partir des entités Java, grâce à l'option `spring.jpa.hibernate.ddl-auto=update`.

Contenu de la table des opérations :

```

tp2_db=# select * from operation;
 id | a | b |      date      | opérateur | resultat
-----+-----+-----+-----+-----+-----
  2 | 10 | 5 | 2026-09-24 01:15:32.622184 | addition | 15
  3 | 10 | 5 | 2026-09-24 01:15:57.494854 | addition | 15
  4 | 10 | 5 | 2026-09-26 10:39:19.033673 | addition | 15
  5 | 10 | 5 | 2026-09-26 10:47:12.004394 | addition | 15
  6 | 10 | 5 | 2026-09-26 10:52:21.279331 | soustraction | 5
  7 | 10 | 5 | 2026-09-26 10:55:58.79487 | division | 2
  8 | 10 | 1 | 2026-09-26 11:22:03.463718 | division | 10
  9 | 10 | 5 | 2026-09-26 11:47:28.809616 | addition | 15
(8 lignes)

```

Exercice 2 : API de conversion de devises avec taux dynamiques

1) Mise en place : GET /api/currency/convert

The screenshot shows a REST client interface with the following details:

- Method:** GET
- URL:** `http://localhost:8080/api/currency/convert?amount=100&from=EUR&to=USD`
- Params:**
 - `amount`: 100
 - `from`: EUR
 - `to`: USD
- Status:** 200 OK
- Response Body (JSON):**

```

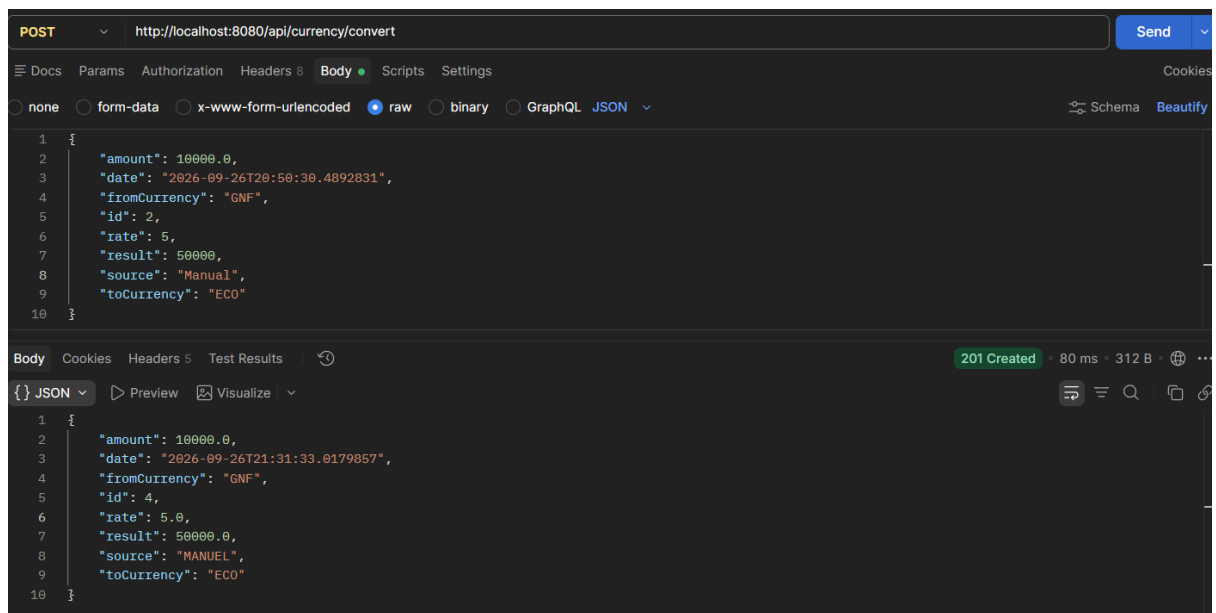
{
  "amount": 100.0,
  "date": "2026-09-26T18:48:13.4815583",
  "fromCurrency": "EUR",
  "id": 1,
  "rate": 1.1403,
  "result": 114.03000000000002,
  "source": null,
  "toCurrency": "USD"
}

```

Le taux de change est récupéré depuis l'API externe Frankfurter, puis la conversion est enregistrée en base. Au départ nous avons utilisé WebClient, mais l'appel restait bloqué avec Java 25 ; nous sommes donc passés à RestClient, le client HTTP synchrone de Spring, qui fonctionne sans problème.

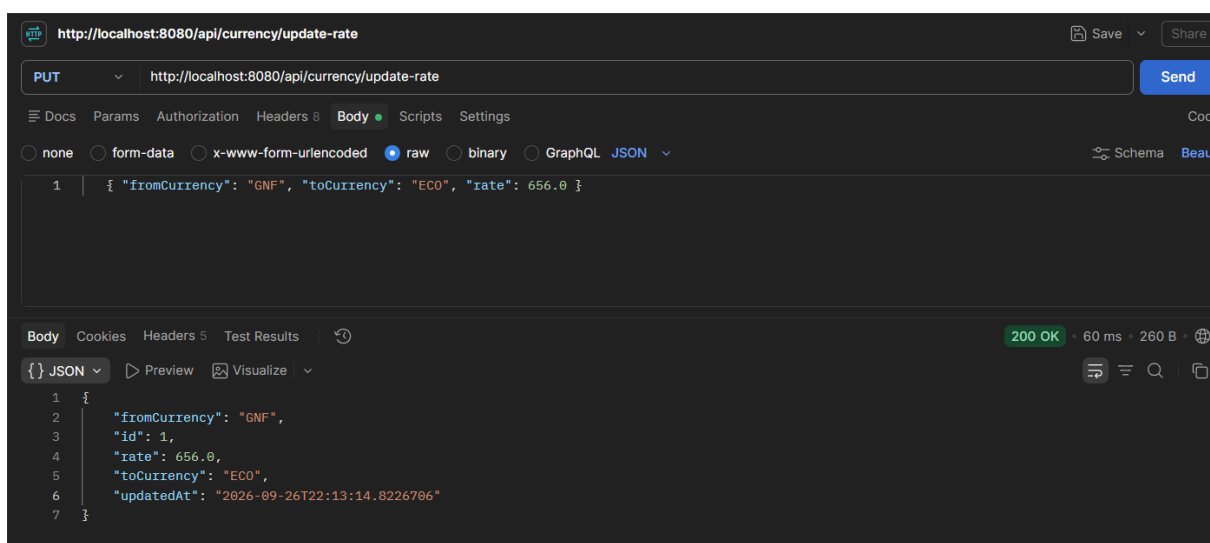
CRUD et robustesse

- 1) POST /api/currency/convert : définit un taux manuel pour des devises non couvertes par l'API externe.

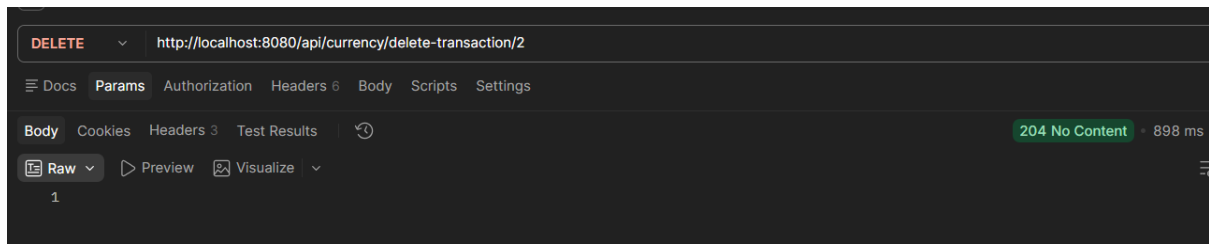


Le taux saisi est enregistré en base, et la transaction est marquée avec la source MANUEL.

- 2) PUT /api/currency/update-rate : met à jour un taux stocké localement.



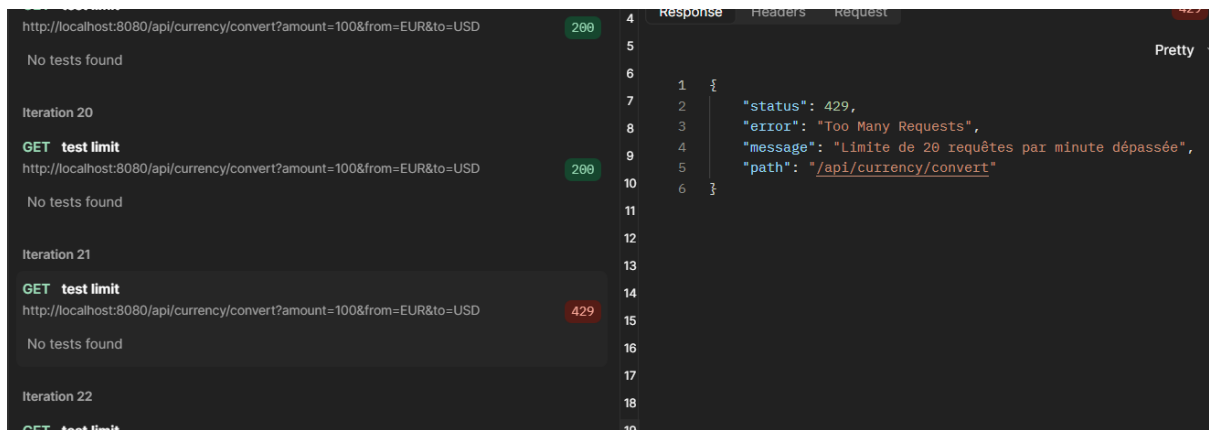
3) DELETE /api/currency/delete-transaction/{id} : supprime une transaction.



La transaction est supprimée et l'API renvoie 204 No Content.

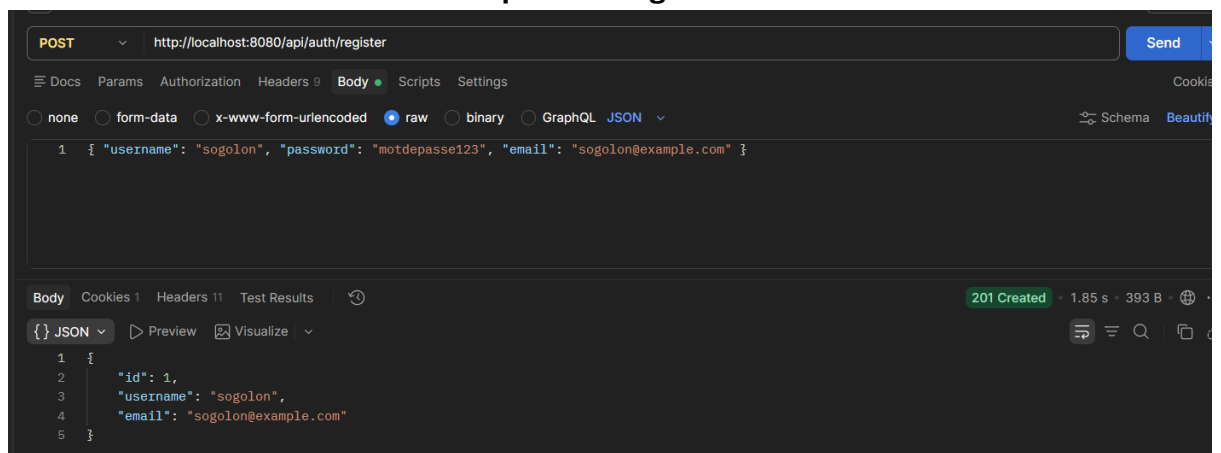
4) Limitation de débit :

Nous avons fixé la limite à 20 requêtes par minute et par adresse IP, avec un filtre Spring personnalisé. Sur la capture du Collection Runner de Postman, on constate que les 20 premières requêtes renvoient 200, et qu'à partir de la 21e requête l'erreur 429 Too Many Requests s'affiche.



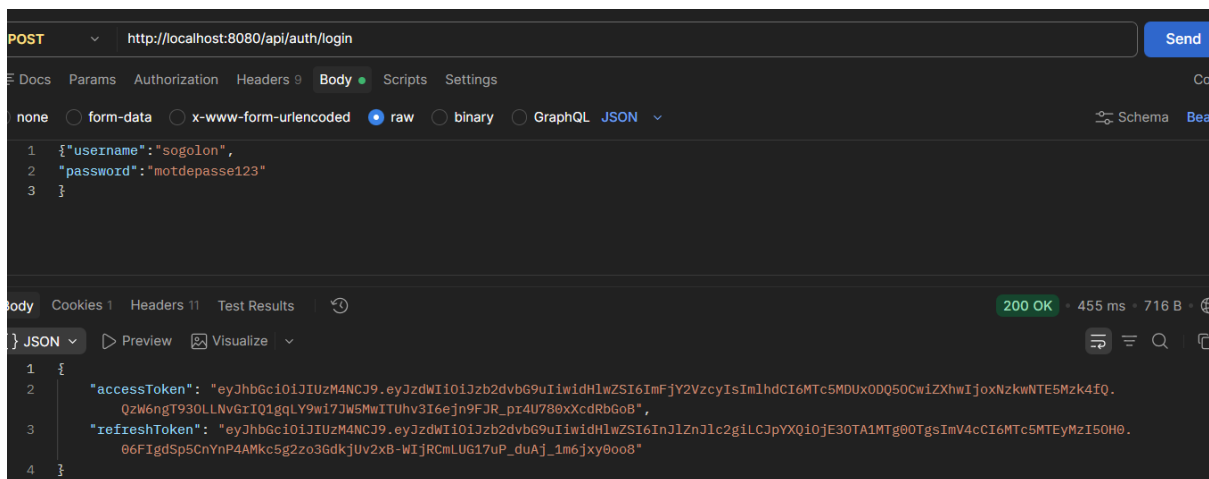
Exercice 3 : API de gestion du BMI avec authentification et statistiques

1) Création d'un utilisateur : POST /api/auth/register



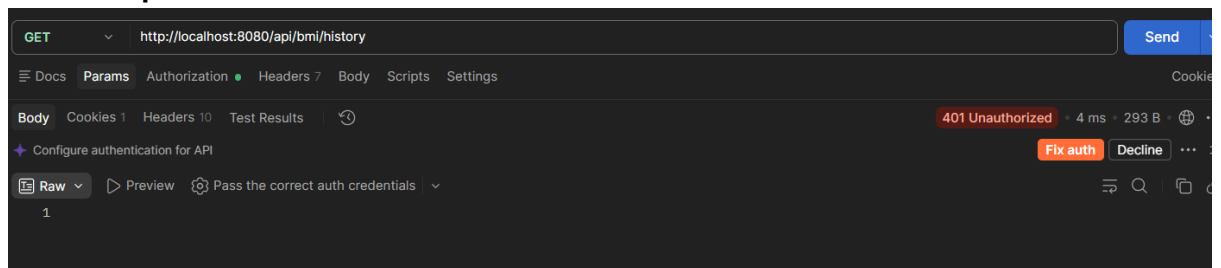
Le mot de passe est haché avec BCrypt avant d'être enregistré en base : il n'est jamais stocké en clair, et il n'apparaît pas dans la réponse.

2) Récupération du token JWT à la connexion : POST /api/auth/login



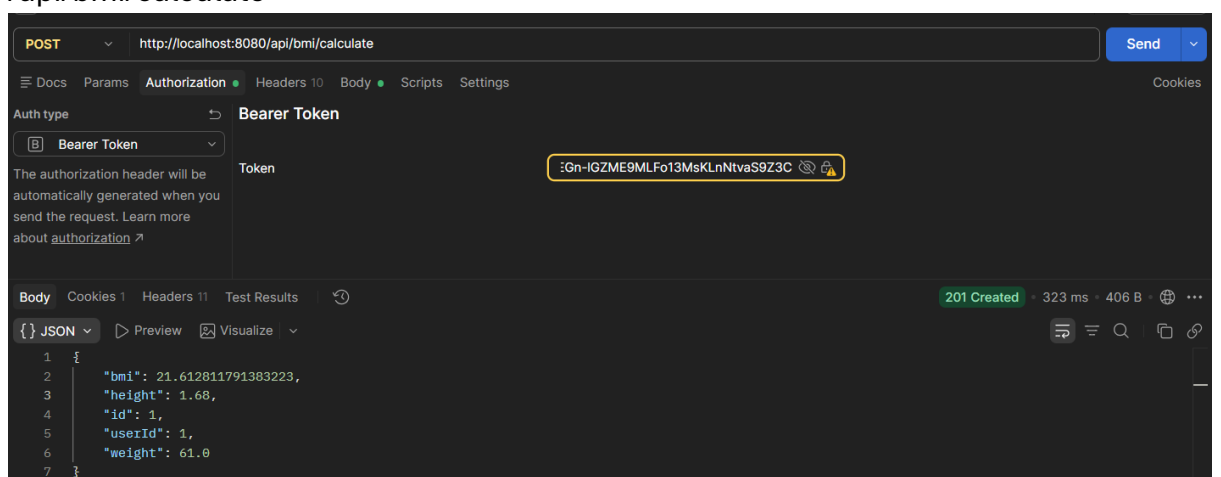
Le token renvoyé a le format `Header.Payload.Signature` vu en cours. Il doit ensuite être envoyé dans l'en-tête « `Authorization: Bearer <token>` » pour accéder aux endpoints BMI.

3) Accès impossible sans token :



Sans en-tête Authorization, l'API renvoie 401 Unauthorized.

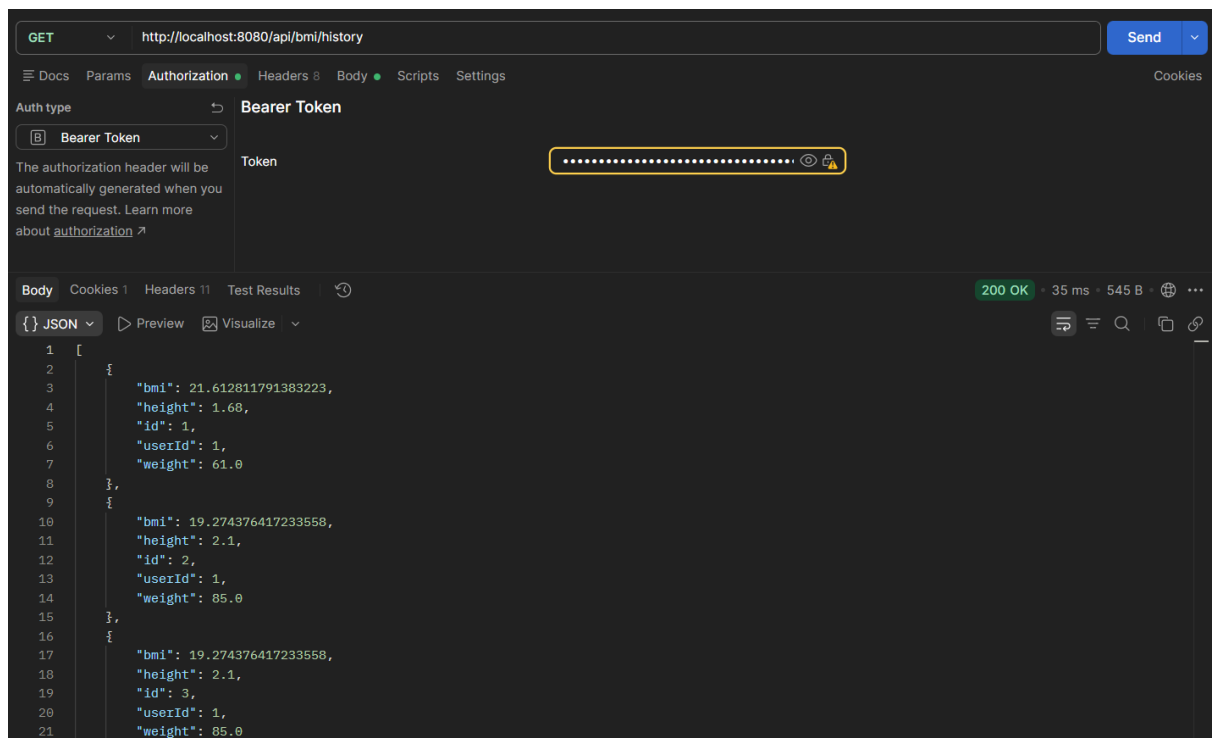
4) Calcul du BMI de l'utilisateur connecté avec un token : POST /api/bmi/calculate



Le BMI est calculé avec la formule poids / taille², la taille étant en mètres (ici 61 / 1,68² ≈ 21,6).

5) Lister les calculs de l'utilisateur connecté :

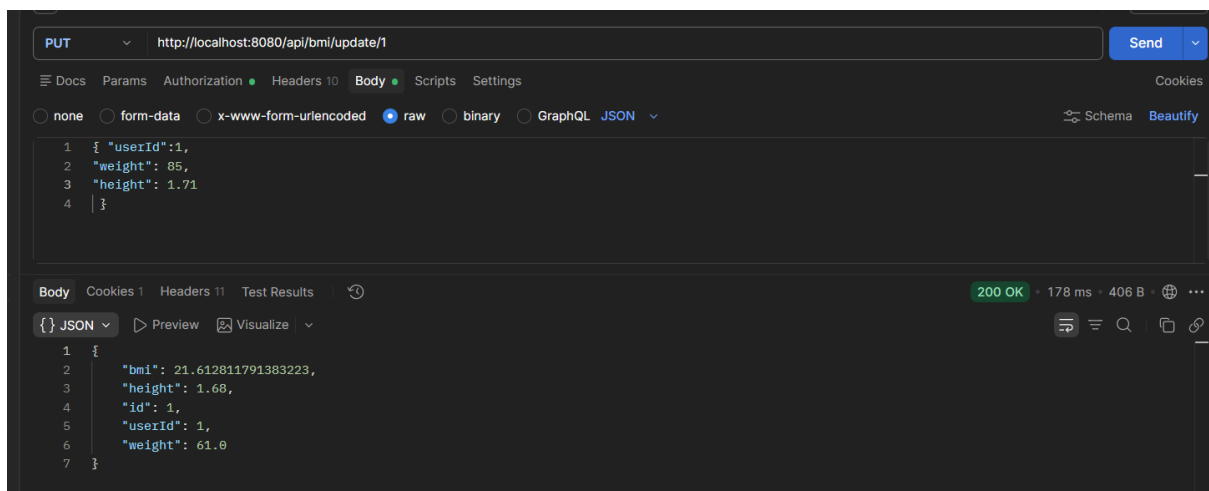
GET /api/bmi/history



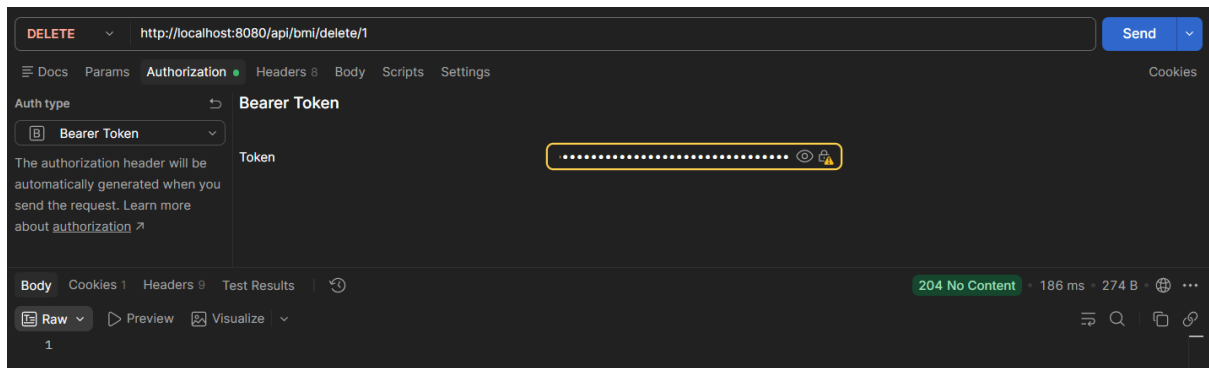
Chaque utilisateur ne voit que ses propres calculs, car il est identifié à partir de son token.

Fonctionnalités avancées

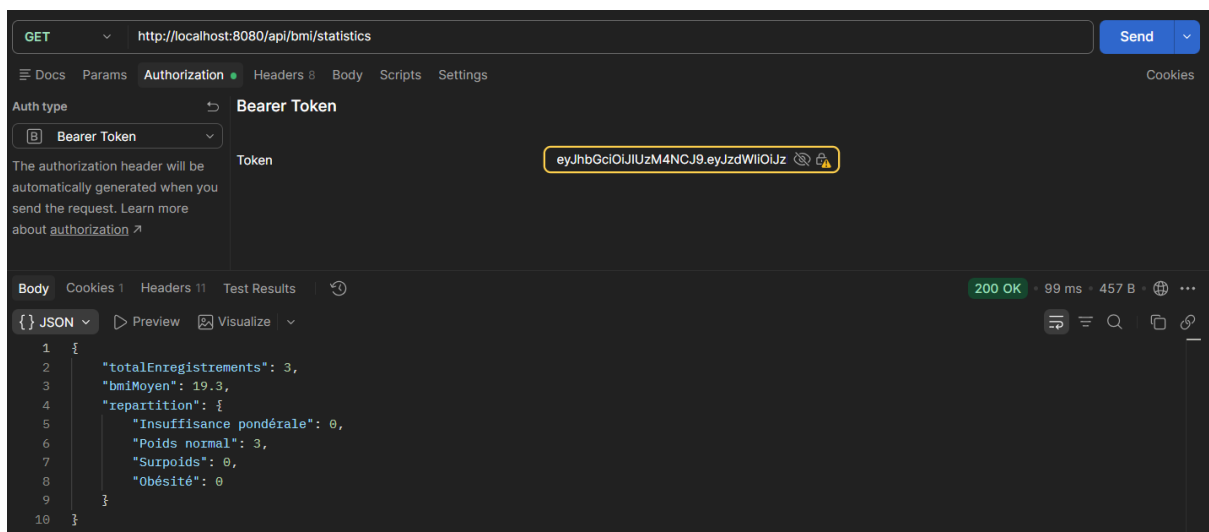
- 1) PUT /api/bmi/update/{id} : met à jour poids/taille d'un enregistrement et recalcule le BMI



- 2) DELETE /api/bmi/delete/{id} : supprime un enregistrement, après avoir vérifié que l'utilisateur authentifié en est bien le propriétaire.



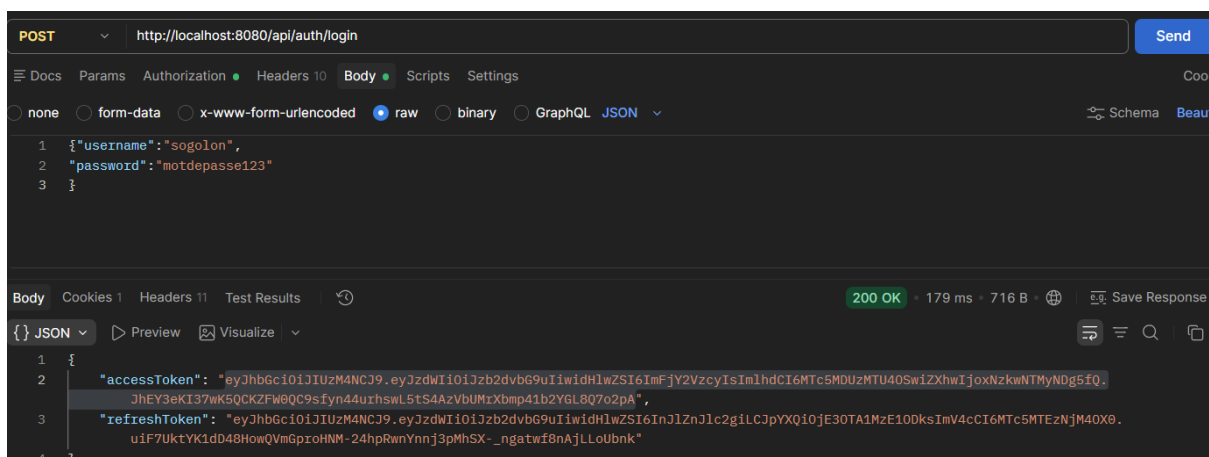
3) GET /api/bmi/statistics : statistiques globales (BMI moyen, répartition par catégorie)



Les catégories utilisées sont celles de l'OMS : insuffisance pondérale, poids normal, surpoids et obésité.

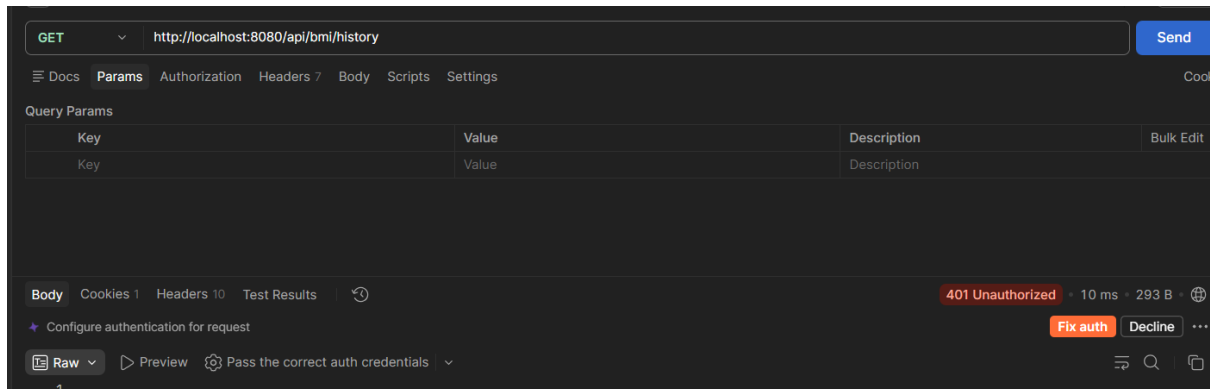
4) Refresh tokens : renouvellement du jeton d'accès sans réauthentification complète de l'utilisateur.

Lors de la connexion (POST /api/auth/login), l'API renvoie deux jetons : un access token de courte durée et un refresh token de plus longue durée. Quand l'access token expire, le client envoie le refresh token à POST /api/auth/refresh pour obtenir un nouvel access token, sans redonner son nom d'utilisateur et son mot de passe.

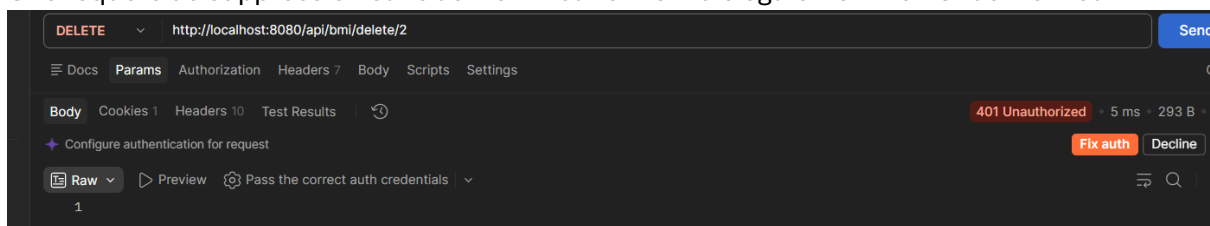


Sécuriser tous les points de terminaison

Une requête qui essaie de lister l'historique des calculs BMI sans authentification renvoie 401 Unauthorized.



Une requête de suppression sans authentification renvoie également 401 Unauthorized :



Conclusion

Ce TP nous a permis de construire une API REST complète avec Spring Boot : persistance avec JPA et PostgreSQL, validation des données, gestion centralisée des erreurs, appel d'une API externe, limitation de débit et sécurisation avec JWT.

Les principales difficultés rencontrées ont été la connexion à PostgreSQL, l'organisation des packages (Spring ne détecte que les classes situées sous le package de la classe principale) et le blocage de WebClient avec Java 25, résolu en passant à RestClient.

Lien github : <https://github.com/Sogolon88/tp2.git>

sur ce lien vous trouverez le json du projet posman ainsi que la structure, code et les ressources du projet.